

## IN DEPTH

**Hello, World!**

Traditionally, the first program a new programmer writes is the “Hello, World!” program. This simply prints the phrase “Hello, World!” on the screen. Even experienced programmers learning a new language often start off in the same way, and it’s also a good first check to see if a newly installed system is working properly. The tradition was introduced in the book *The C Programming Language*, published in 1978.

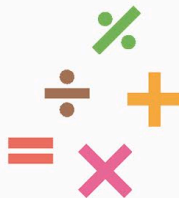
```
class Ball:
    colour = ""
    size = 0
    def throw(self):
        print("ball being thrown!")
    def catch(self):
        print("ball being caught!")
myball = Ball()
myball.colour = "red"
myball.size = 5
```



Python

```
fac 0 = 1
fac n = n * fac (n-1)

main = print (fac 7)
```



Haskell

**Object-oriented**

This style of programming includes the concept of objects that model real-world things. An object usually has fields (containing data) and methods, (containing code) that represent behaviours. So, a ball object might have the fields colour and size, since these are characteristics of a ball, and the method bounce, since this is what a ball does. Objects are instances of classes, definitions of what a particular object would look like. This means the object ball is an instance of the class ball, similar to any real ball being an instance of the idea of a ball.

**Functional**

Functional languages define a program as a series of mathematical functions. A functional language is described as pure if it doesn't affect the computer's state, or impure if it does. One major feature of functional languages is that they don't use loops to repeat operations. Instead, they use a recursive function, which calls itself as part of its own definition. Another notable feature is pattern matching, where a function decides what to do by looking at the value it's been given and seeing which of the several patterns it matches.

## REAL WORLD

**Natural-language programming**

There are programming languages where the code looks like normal text or natural language. However, these aren't serious languages, as even tiny calculations take a large amount of code, and they're usually created just for fun. These include Shakespeare, where programs look like very confused Shakespearean plays, and Chef, where each program is written as a cooking recipe.